Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 233 (2024) 812–821

Procedia Computer Science

www.elsevier.com/locate/procedia

5th International Conference on Innovative Data Communication Technologies and Application (ICIDCA 2024)

BiT5: A Bidirectional NLP Approach for Advanced Vulnerability Detection in Codebase

Prabith GS[c,a,*], Rohit Narayanan M[d,a], Arya A[e,a], Aneesh Nadh R[f,a], Binu PK[g,b]

[a]*Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri, India*
[b]*Department of Computer Science and Applications, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri, India*
[c]*amenu4cse20351@am.students.amrita.edu*
[d]*amenu4aie20160@am.students.amrita.edu*
[e]*amenu4aie20114@am.students.amrita.edu*
[f]*amenu4eac20010@am.students.amrita.edu*
[g]*binu@am.amrita.edu*

Abstract

In this research paper, a detailed investigation presents the utilization of the BiT5 Bidirectional NLP model for detecting vulnerabilities within codebases. The study addresses the pressing need for techniques enhancing software security by effectively identifying vulnerabilities. Methodologically, the paper introduces BiT5, specifically designed for code analysis and vulnerability detection, encompassing dataset collection, preprocessing steps, and model fine-tuning.

The key findings underscore BiT5's efficacy in pinpointing vulnerabilities within code snippets, notably reducing both false positives and false negatives. This research contributes by offering a methodology for leveraging BiT5 in vulnerability detection, thus significantly bolstering software security and mitigating risks associated with code vulnerabilities.

© 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application

Keywords: Bidirectional Transformer; BiT5 Model; Code Analysis; Code Vulnerabilities; Machine Learning; Natural Language Processing (NLP); Software Security; Vulnerability Detection;

1. Introduction

In the world of software systems ensuring security is a crucial concern. The presence of weaknesses, in software codes, poses a risk to the stability and safety of these systems. The seriousness of these threats cannot be overstated, in sectors like finance, healthcare, and infrastructure. These vulnerabilities serve as potential gateways for cyber threats,

1877-0509 © 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application
10.1016/j.procs.2024.03.270

enabling data breaches and unauthorized access that can wreak havoc, causing substantial financial losses, compromising the confidentiality of sensitive information in sectors like finance and healthcare, and disruptive breakdowns in critical infrastructure. The gravity of these threats cannot be overstated; they hold the potential to induce profound societal and economic repercussions, emphasizing the criticality of a robust approach to identifying and mitigating these vulnerabilities.

Vulnerabilities in software systems come in forms such as buffer overflows, injection attacks, and logical flaws. Each type of vulnerability brings its set of challenges by exploiting weaknesses that compromise the confidentiality and integrity of information. Additionally, with cyber threats evolving rapidly, it is essential to take an approach to identify these vulnerabilities.

1.1. Emerging Need for Advanced NLP Techniques

The convergence of Natural Language Processing (NLP) and code analysis brings hope to vulnerability detection[1]. NLP techniques provide an understanding of the language structures embedded in code going beyond the limitations of syntax based analysis. Directional NLP models, like BiT5 introduce an approach by capturing contextual information from both preceding and succeeding code segments[2].

The intricate relationship between software vulnerabilities the challenges faced by detection methods and the rise of NLP techniques sets the stage for an evolutionary step, towards safeguarding software systems[3, 4].

For instance, models like CodeT5+ showcase an advanced understanding of code structures, allowing accurate identification of vulnerabilities across various programming languages. This capability surpasses traditional syntax-based approaches, enabling nuanced context comprehension and precise vulnerability pinpointing within code snippets. The robustness of these models in capturing contextual information from preceding and succeeding code segments further exemplifies their potential to fortify software security measures, marking a pivotal advancement in safeguarding against cyber threats[2].

1.2. Introduction to BiT5: A Bidirectional NLP Model

The BiT5 model's strengths encompass its Transformer-based architecture with self-attention mechanisms, facilitating robust relationship capture within code snippets by considering the token context. Its distinctive bidirectional nature allows holistic comprehension of code semantics across subsequent segments[5, 6, 7], enhancing contextual understanding within code segments. Moreover, leveraging token embeddings, BiT5 transforms raw code sequences into dimensional representations, adeptly capturing syntax, and semantic structures, crucial for precise vulnerability detection[8]. However, potential limitations might involve computational resource intensiveness due to its transformer-based architecture and a need for substantial labeled data for effective fine-tuning, restricting applicability in scenarios with limited labeled datasets.

1.2.1. Application Scope in Vulnerability Detection

BiT5 possesses capabilities that make it an invaluable asset, in the field of code analysis especially when it comes to identifying vulnerabilities within software systems[9]. The models ability to analyze bidirectionally gives it an advantage allowing it to recognize patterns and subtle cues that indicate vulnerabilities. By considering the context of both preceding and succeeding parts of code BiT5 captures the essence of code segments comprehensively greatly improving vulnerability detection accuracy.

Additionally BiT5s proficiency in handling transformations that preserve the meaning of code addresses a standing challenge in vulnerability detection[10]. These transformations are often used to hide vulnerabilities. Have proven difficult for methods. However thanks to its understanding of code semantics BiT5 is able to navigate through these transformations reducing the risks associated with false positives or negatives.

The adaptability of BiT5 across programming languages further enhances its effectiveness. Its architecture is not dependent on syntax intricacies allowing it to transcend language barriers and serve as a tool for vulnerability detection, in various codebases.

2. Literature Review

2.1. Traditional Methodologies for Vulnerability Detection

When it comes to software security there have been methods, like Dynamic analysis which observe code behavior during execution but may not cover all scenarios comprehensively, Pattern matching and signature-based detection rely on predefined patterns, which can limit their effectiveness against emerging or new threats that aim to find vulnerabilities in codebases. Static analysis examines code structure without execution, potentially missing complex runtime issues. Dynamic analysis observes runtime behavior but may struggle to cover all scenarios. Pattern matching and signature-based detection rely on predefined patterns, making them less effective against new threats.

2.2. Limitations of Conventional Approaches

Traditional approaches in vulnerability detection often grapple with overfitting, constraining their adaptability to diverse code structures and languages. Their inherent inflexibility with programming languages hampers their effectiveness across environments. Real-time deployment complexities restrict their efficiency in dynamic software systems. Scalability concerns further limit their capacity to comprehensively detect vulnerabilities, especially within extensive codebases or large-scale systems.

Traditional approaches, like pattern matching, struggle to adapt to evolving threats. For instance, signature-based detection relies on predefined patterns, making it less effective against new attack variations. In contrast, NLP-based models, such as BiT5, excel in understanding nuanced code semantics, enabling the detection of vulnerabilities beyond predefined patterns. Additionally, traditional methods often face scalability issues when handling extensive codebases, limiting their comprehensive vulnerability detection across large-scale systems. In contrast, NLP techniques leverage advanced models like BiT5 to efficiently analyze diverse codebases, enhancing scalability and adaptability in dynamic software ecosystems.

2.3. Recent Advancements in NLP-Based Vulnerability Detection

Lately there have been advancements, in using NLP models to find weaknesses in codebases. BiT5 and CodeT5+ [11] are examples of NLP models that have shown skills in understanding the meaning of code and putting it into context[11]. They outperform vulnerability detection methods by uncovering connections, within code snippets, which improves their ability to identify vulnerabilities accurately and precisely.

2.4. Comparative Analysis of NLP-Driven Techniques

Comparing approaches, in Natural Language Processing (NLP) requires scrutinizing performance metrics like accuracy, precision, recall, and F1 score across various models to assess their effectiveness in detecting vulnerabilities. Our evaluation, spanning diverse codebases and programming languages, provides insights into the strengths and limitations of each model, helping us identify areas of excellence and opportunities for improvement in their adaptability and robustness when dealing with code structures.

2.5. Current Challenges and Future Directions

Although there have been advancements, in NLP based vulnerability detection there are still challenges that need to be addressed. One of the obstacles is ensuring that models can effectively handle diverse codebases as this is crucial, for their overall performance. Another challenge lies in language diversity as models often struggle to overcome limitations when dealing with programming languages[3]. Additionally the real time applicability of these models is hindered by demands making it difficult to seamlessly integrate them into software environments. For instance, a model trained on Python might struggle when faced with C++ code due to syntactic and semantic disparities. Moreover, ensuring real-time applicability of these models remains a challenge, given the demands for seamless integration into dynamic software environments. This hurdle involves balancing computational demands with the need for swift and accurate vulnerability detection within live systems.

3. Methodology

In the domain of identifying vulnerabilities, in software systems the process of acquiring and preparing datasets plays a role in training Natural Language Processing (NLP) models[12]. This section explores the methods used to gather and refine datasets that're indispensable for vulnerability detection tasks.

3.1. Dataset Collection and Preparation Techniques

The search for datasets begins with an approach that involves various sources and careful curation methods. Repositories such as the Software Assurance Reference Dataset (SARD) BigVuln, National Vulnerability Database (NVD) and platforms like GitHub serve as resources offering a range of code snippets that encompass diverse vulnerabilities[12].

SARD- A Goldmine of Vulnerabilities: SARD is renowned for its compilation of software flaws and vulnerabilities providing an abundance of real world code snippets. This repository is meticulously curated by the National Institute of Standards and Technology (NIST). Covers vulnerabilities, in programming languages and domains. Leveraging SARD enhances the dataset with a variety of vulnerabilities enabling a training set with content.

BigVuln- Augmenting Diversity and Scale: The BigVuln dataset complements the effort by providing a collection of code snippets that address vulnerabilities. This compilation includes advanced vulnerabilities expanding the datasets depth and breadth. By including BigVuln we enhance the diversity and complexity, within the dataset ensuring that the NLP model can effectively comprehend vulnerabilities.

NVD and GitHub Repositories- Real-World Insights: The National Vulnerability Database (NVD) and GitHub repositories are resources that offer a real world view of vulnerabilities. NVD is known as a repository of vulnerability related information encompassing vulnerabilities reported in software products thus providing an understanding of real world vulnerabilities. GitHub, being a hub for codebases contributes up to date information on vulnerabilities observed in actively developed projects.

For instance, SARD encapsulates vulnerabilities such as buffer overflows, SQL injections, and cross-site scripting (XSS) attacks. BigVuln, on the other hand, extends this diversity by including more sophisticated vulnerabilities like zero-day exploits or race conditions, augmenting the dataset's complexity. Moreover, repositories like NVD and GitHub contribute by providing insights into real-world vulnerabilities observed in software products and actively developed projects, covering instances such as directory traversal, command injection, or code injection attacks, thus exposing NLP models to a wide spectrum of vulnerabilities prevalent across various domains and programming languages, essential for comprehensive vulnerability detection.

The amalgamation of datasets from various repositories, coupled with meticulous preprocessing techniques, bequeaths a robust foundation for training advanced NLP models. This curated dataset, rich in diversity and quality, forms the linchpin for enhancing the efficacy of vulnerability detection using Bidirectional NLP models like BiT5[13, 14].

3.2. Preprocessing Techniques for Dataset Refinement

3.2.1. Data Cleaning for Uniformity

Standardizing Code Snippets:

Aligning code snippets to a structure is a step in achieving uniformity across the dataset. This process involves removing variations in coding styles, indentation and formatting ensuring that the dataset presents a representation of code logic to enhance the models understanding of code semantics.

Removal of Redundant Elements

It is crucial to eliminate comments, excessive whitespace or non executable code segments within the snippets. By doing we can streamline the snippets. Focus on essential code segments that are relevant to vulnerability detection enabling the model to accurately identify vulnerabilities[13, 15, 14].

3.2.2. Formatting and Labeling

Code Tokenization

The process of dissecting code snippets into units or tokens, aids in enhancing our comprehension of their syntactic and

semantic structures. This method boosts the model's capability to identify vulnerability-related patterns by breaking down the code into smaller, more manageable components

Uniform Labeling Conventions

Having labeling conventions is essential for clarity and coherence within the dataset. Ensuring labels for non vulnerable code segments facilitates the learning process of the model and enables accurate classification of vulnerabilities.

3.2.3. Data Consistency and Quality Assurance

Quality Control Measures

Implementing quality control measures is crucial, for maintaining data consistency throughout all stages of preprocessing. Ensuring the quality of the dataset is a step, in the validation process. By using checking and verification techniques we can mitigate any inclusion of incorrect or mislabeled data instances thus preserving the integrity of the dataset.

Balancing the Dataset

Another important aspect is to balance the dataset[16, 17]. It's essential to have a representation of both non vulnerable instances. Imbalanced datasets can introduce biases and negatively impact model learning. Techniques such, as oversampling or undersampling can be employed to achieve a distribution, which promotes unbiased model training.

3.3. BiT5 Architecture and Suitability for Vulnerability Detection

3.3.1. BiT5 Architecture Overview

Bidirectional Transformer Framework

Self Attention Mechanisms: The core element of BiT5 lies in its self attention mechanisms, which allow the model to simultaneously weigh and contextualize parts of the code snippet. By utilizing head attention layers BiT5 assesses dependencies among code tokens bidirectionally enabling a comprehensive understanding of the context in which the code operates.

Encoder-Decoder Paradigm: BiT5 employs an encoder decoder architecture that facilitates processing. This architecture not captures the sequence relationships within the code but also incorporates a decoding mechanism to generate meaningful representations that encompass both directions of the codebases context.

3.3.2. Bidirectional Code Processing

Contextual Embeddings: excels at generating embeddings for code tokens by encoding syntactic and semantic information bidirectionally. This approach enables a nuanced comprehension of the aspects in the code. By considering both directions this bidirectional encoding strategy enhances the models ability to capture patterns and nuances in code semantics.

Semantic Understanding: Through its integration of processing BiT5 achieves an understanding of code semantics by considering relationships between different segments in both directions. This holistic interpretation allows for an analysis of factors such as code structure, functions and interactions, between variables, methods or classes.

3.3.3. Suitability for Vulnerability Detection

When it comes to identifying vulnerabilities BiT5 excels due, to its ability to understand the nuanced connections and dependencies present in code snippets. This comprehensive understanding allows the model to uncover vulnerabilities that may be hidden within code structures or disguised by variations, in semantics.

Handling Diverse Codebases BiT5 possesses the ability to handle codebases effectively. It can easily accommodate syntaxes, semantics and programming languages which helps overcome the challenges associated with code diversity that exist in real world software systems[3].

3.4. Applicability of BiT5 in Vulnerability Detection

3.4.1. Bidirectional Understanding of Code Structures

Comprehensive Contextualization: Thanks, to its nature BiT5 can grasp code structures comprehensively by placing code snippets in their contexts. It captures dependencies and relationships within the code identifying nuances that may indicate vulnerabilities.

Semantic Context Analysis: By considering context BiT5 understands the implications of different parts of code. It recognizes the multifaceted relationships between these elements allowing it to identify vulnerabilities that might go unnoticed by models.

3.4.2. Addressing Traditional Method Challenges

Overcoming Overfitting: Through its processing approach BiT5 reduces overfitting problems by taking into account context instead of relying solely on isolated code segments. This improves model robustness. Enables generalization across diverse codebases.

Handling Language Diversity: BiT5s bidirectional nature allows it to transcend language barriers and adapt to syntaxes and semantics across programming languages. It possesses an understanding of code regardless of language constraints giving it a significant advantage, over traditional methods.

3.4.3. Enhanced NLP Capabilities

Advanced Semantic Understanding: BiT5 possesses natural language processing capabilities allowing it to comprehend the semantics of code more deeply. This enables an analysis of code structures and enhances the accuracy of detecting vulnerabilities.

Contextualized Vulnerability Identification: By conducting analysis BiT5 contextualizes potential vulnerabilities, within the larger code context reducing the occurrence of false positives and negatives. This contextual understanding ensures a identification of vulnerabilities.

3.5. Fine-Tuning BiT5 for Vulnerability Detection

3.5.1. Initialization of BiT5 Model

Model Configuration for Vulnerability Detection Configuring the trained BiT5 model for vulnerability detection tasks is a crucial step during the initialization phase. One key aspect is modifying the models output layer to cater to classification, where it distinguishes between non vulnerable code segments. This entails adjusting the output layer structure to align with the requirements of identifying vulnerabilities.

3.5.2. Fine-Tuning Process

Training Methodology: The tuning process aims to optimize BiT5s performance in vulnerability detection by utilizing a dataset specifically prepared for this task[18, 18]. Through training the model adapts its weights and parameters to effectively identify vulnerabilities within code snippets.

Loss Function Selection: To guide the tuning phase a binary cross entropy loss function is employed. This function is selected due to its effectiveness in facilitating the learning process for classification tasks. It enables BiT5s parameters to be optimized accurately classifying code segments as either vulnerable or non vulnerable.

Hyperparameter Tuning: Optimizing hyperparameters plays a role in tuning. Parameters such, as learning rate batch size and number of training epochs are adjusted meticulously to achieve optimal model performance. By adjusting these hyperparameters we can ensure that converges reducing loss and improving accuracy, in vulnerability detection.

Weight Optimization Strategies: To tune the models weights we employ techniques such as gradient descent based methods, regularization and adaptive learning rates. These approaches help refine BiT5s weights through a process ultimately enhancing its capability to accurately detect vulnerabilities, in code snippets.

4. Experimentation and Results

4.1. Experimental Setup for BiT5 Training

The experiments were carried out using Google Colab, which's a platform, in the cloud that allows for the execution of Python code. The hardware setup included a Cloud Compute Engine with specifications such as an 8 core CPU, 16 threads, a GPU with 4GB of VRAM and either 16GB or 32GB of RAM. Various software tools were utilized including TensorFlow, PyTorch, Keras, Scikit learn, BeautifulSoup, Scrapy, Pandas, Transformers from Hugging Face and PyTorch Lightning[19, 20].

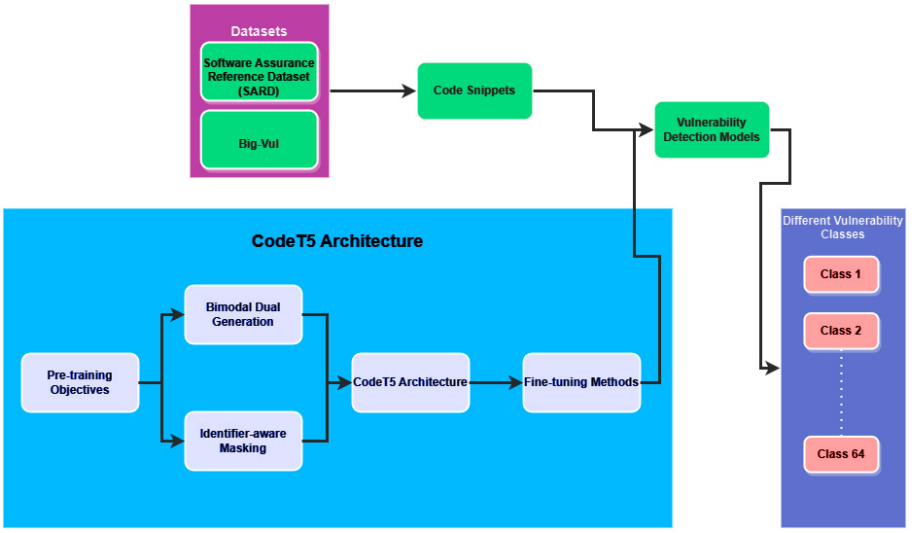


Fig. 1. Proposed Modules

4.1.1. Types of Experiments Conducted

During the phase a series of analyses were conducted to evaluate the effectiveness of vulnerability detection models. Each experiment focused on refining and optimizing models using techniques and configurations, such as fine-tuning hyperparameters, adjusting learning rates, and employing specific loss functions.

4.1.2. Experiment 1: Fine Tuning CodeT5+ Model

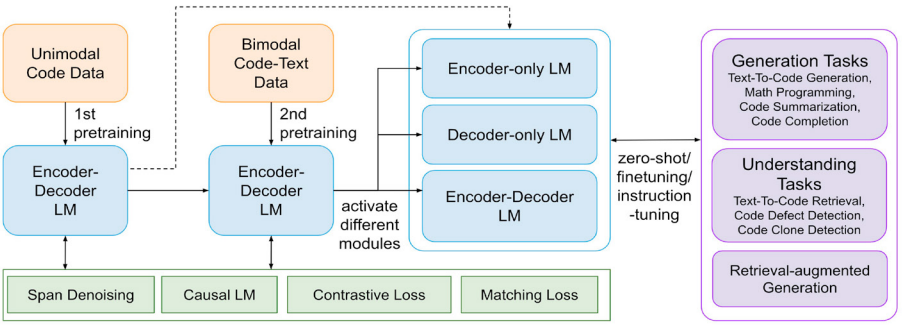


Fig. 2. CodeT5+ Model Architecture

Performance Metrics: The experiment showcased enhancements, in accuracy, precision, recall rates well as F1 Score when compared to baseline models.

Model Training Performance Analysis: We observed accuracy and recall indicating that the model effectively captures vulnerabilities. However there is potential, for improving precision to reduce positives.

Inference and Explanation The model did a job at identifying vulnerabilities. Could benefit from further optimization to decrease false positives while still maintaining its ability to detect actual vulnerabilities.

4.1.3. Experiment 2: Comparative Analysis of Vulnerability Detection Models

Models Under Scrutiny

DeepWukong Model utilizes a Graph Neural Network (GNN) integrated with deep learning methodologies. Operating on Extracted XFGs from Program Dependence Graphs (PDGs), it distinguishes vulnerable from non-vulnerable

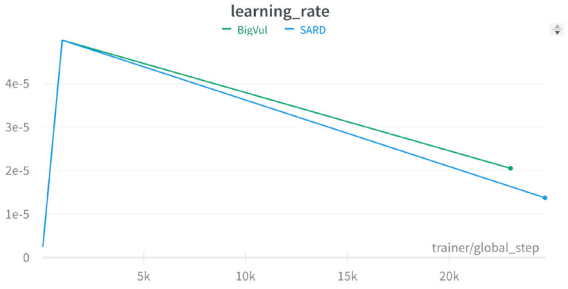


Fig. 3. Learning Rate

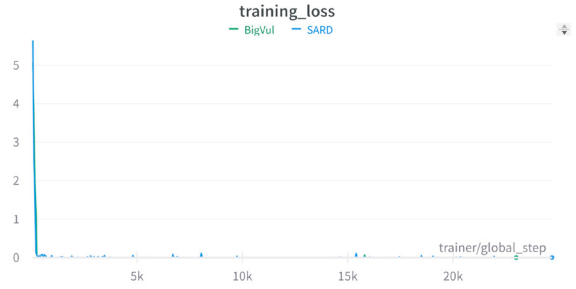


Fig. 4. Training Loss

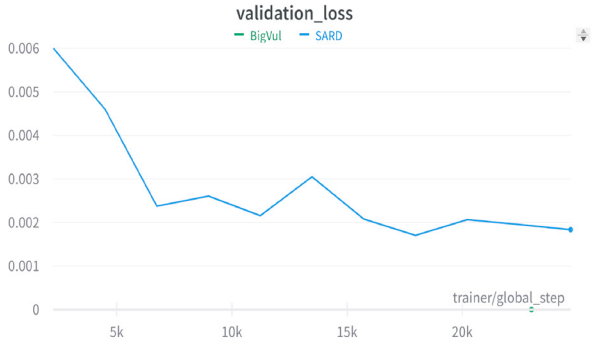


Fig. 5. Validation Loss

code segments. A pivotal feature involves the identification of vulnerable code statements by employing forward and backward slicing techniques [21].

LineVul Model relies on a Deep Learning framework leveraging CodeBERT. Its input comprises tokenized source code statement chunks, enabling the model to classify segments as vulnerable or non-vulnerable. Multi-head self-attention layers and a Dense neural network form the core components, empowering the model to pinpoint vulnerable lines within the source code.

BiT5 (CodeT5) adopts a Transformer-based architecture (CodeT5+). Analyzing source code snippets, it distinguishes code segments as either vulnerable or non-vulnerable.

4.2. Comparative Analysis

The comparative evaluation among the DeepWukong, LineVul, and BiT5 models reveals insightful performance differences in vulnerability detection within codebases:

Table 1. Performance Metrics of Models

Model	**Accuracy**	**Precision**	**Recall**	**F1-Score**
DeepWukong	95%	87%	96%	93%
LineVul	96%	96%	84%	90%
BiT5	97.47%	95.32%	98.05%	96.66%

4.2.1. Accuracy Analysis

Code-T5 demonstrates the highest accuracy at 97.47%, closely followed by LineVul at 96%, while DeepWukong achieves 95% accuracy.

4.2.2. Precision and Recall Trade-offs

DeepWukong excels in recall detecting 96% of vulnerabilities. However it demonstrates precision at 87%, which means it may have some false positives. LineVul achieves a precision of 96% and recall of 84% implying it strikes a balance, between correctly identifying vulnerabilities and minimizing false positives though it might miss some actual vulnerabilities in the process. Code T5 exhibits balance with a precision rate of 95.32% and recall rate of 98.05%. This indicates that it accurately identifies vulnerabilities while maintaining a rate of both positives and negatives.

4.2.3. F1 Score Analysis

Among the vulnerability detection models Code T5 achieves the F1 Score of 96.66% indicating its ability to balance precision and recall effectively. DeepWukong achieves an F1 Score of 93% suggesting that it identifies vulnerabilities compared to Code T5. LineVul achieves a F1 Score of 90% demonstrating a balance, between precision and recall albeit with a slight trade off.

The comparative assessment reveals differences in vulnerability detection models. Code T5 demonstrates performance by balancing precision and recall. DeepWukong prioritizes recall, while LineVul strikes a balance but leans slightly towards precision. In conclusion Code T5 is chosen as the model for research in this field due to its overall performance.

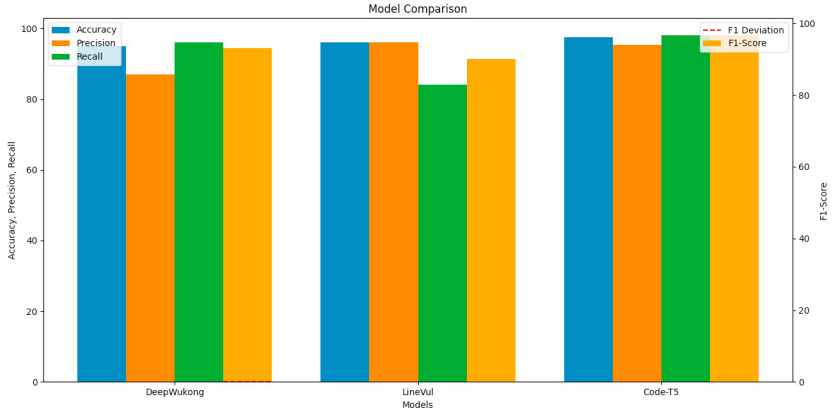


Fig. 6. Model Comparison Metrics

5. Conclusions

The research contributes to the field of cybersecurity and software development in several key ways:
Advanced Vulnerability Detection: By employing state-of-the-art transformer-based models and leveraging large-scale datasets, the study advances the sophistication and accuracy of vulnerability detection methodologies.
Optimized Model Training: The experiments provided insights into fine-tuning models, showcasing techniques to enhance accuracy while minimizing false positives in vulnerability identification.

The research efforts were focused on optimizing vulnerability detection using transformer-based models like CodeT5+. Through meticulous fine-tuning, the CodeT5+ model exhibited remarkable enhancements in accuracy, showcasing a significant increase from 94% to 97.47%, and recall, rising from 93% to 98%. These improvements were pivotal in ensuring the model's adeptness at precisely identifying vulnerabilities within code snippets. While the study emphasizes the positive advancements in vulnerability detection methodologies by leveraging transformer-based models and augmented datasets, it is crucial to acknowledge potential limitations. Transformer-based models might still face challenges in handling semantic-preserving code transformations or nuanced vulnerabilities embedded within complex code structures. Moreover, ensuring the scalability of these models to accommodate diverse codebases across multiple programming languages remains an ongoing challenge.

The implications of this research for cybersecurity and software development are significant. The utilization of advanced transformer-based models and expanded datasets marks a pivotal advancement in vulnerability detection, offering more accurate and sophisticated methods to identify and mitigate potential threats in software systems. However, the field continues to evolve, necessitating ongoing exploration and refinement of these techniques to address emerging challenges and ensure robust security measures in software development and cybersecurity practices.

5.1. Future Scope

Future research endeavors should prioritize the development of transformer-based models adept at handling a broader spectrum of programming languages, expanding beyond conventional ones like C/C++ and Java. Focus on refining models to effectively tackle semantic-preserving transformations and intricacies within complex code structures would substantially enhance their robustness. Augmenting datasets with a diverse array of samples, encompassing varied vulnerabilities across different programming paradigms, will be instrumental in fortifying model accuracy and generalization capabilities. Additionally, exploring techniques to mitigate biases in datasets and models while ensuring scalability across large-scale systems remains a critical avenue for further investigations.

References

[1] M. Omar, "Detecting software vulnerabilities using language models," *arXiv preprint arXiv:2302.11773*, 2023.

[2] W. Huang, S. Lin, and L. Chen, "Bbvd: A bert-based method for vulnerability detection," *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 12, 2022.

[3] N. Risse and M. Böhme, "Limits of machine learning for automatic vulnerability detection," *arXiv preprint arXiv:2306.17193*, 2023.

[4] N. Ziems and S. Wu, "Security vulnerability detection using deep learning natural language processing," in *IEEE INFOCOM 2021-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 2021, pp. 1–6.

[5] B. S. Kumar and K. Pujitha, "Web application vulnerability detection using hybrid string matching algorithm," *vol*, vol. 7, pp. 106–109, 2018.

[6] D. Gupta, K. Vani, and C. K. Singh, "Using natural language processing techniques and fuzzy-semantic similarity for automatic external plagiarism detection," in *2014 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*. IEEE, 2014, pp. 2694–2699.

[7] B. Premjith, M. A. Kumar, and K. Soman, "Neural machine translation system for english to indian language translation using mtil parallel corpus," *Journal of Intelligent Systems*, vol. 28, no. 3, pp. 387–398, 2019.

[8] B. Steenhoek, M. M. Rahman, R. Jiles, and W. Le, "An empirical study of deep learning models for vulnerability detection," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2237–2248.

[9] S. Jose, K. Priyadarshini, and K. Abirami, "An analysis of black-box web application vulnerability scanners in sqli detection," in *Proceedings of the International Conference on Soft Computing Systems: ICSCS 2015, Volume 2*. Springer, 2016, pp. 177–185.

[10] K. Napier, T. Bhowmik, and S. Wang, "An empirical study of text-based machine learning models for vulnerability detection," *Empirical Software Engineering*, vol. 28, no. 2, p. 38, 2023.

[11] Y. Wang, H. Le, A. D. Gotmare, N. D. Bui, J. Li, and S. C. Hoi, "Codet5+: Open code large language models for code understanding and generation," *arXiv preprint arXiv:2305.07922*, 2023.

[12] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner, "Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection," in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*, 2023, pp. 654–668.

[13] C. Zhang, H. Liu, J. Zeng, K. Yang, Y. Li, and H. Li, "Prompt-enhanced software vulnerability detection using chatgpt," *arXiv preprint arXiv:2308.12697*, 2023.

[14] W. Tang, M. Tang, M. Ban, Z. Zhao, and M. Feng, "Csgvd: A deep learning approach combining sequence and graph embedding for source code vulnerability detection," *Journal of Systems and Software*, vol. 199, p. 111623, 2023.

[15] W. Jie, Q. Chen, J. Wang, A. S. V. Koe, J. Li, P. Huang, Y. Wu, and Y. Wang, "A novel extended multimodal ai framework towards vulnerability detection in smart contracts," *Information Sciences*, vol. 636, p. 118907, 2023.

[16] T. Bikku, J. Jarugula, L. Kongala, N. D. Tummala, and N. V. Donthiboina, "Exploring the effectiveness of bert for sentiment analysis on large-scale social media data," in *2023 3rd International Conference on Intelligent Technologies (CONIT)*. IEEE, 2023, pp. 1–4.

[17] G. Bharathi Mohan, R. Prasanna Kumar, S. Parathasarathy, S. Aravind, K. Hanish, and G. Pavithria, "Text summarization for big data analytics: A comprehensive review of gpt 2 and bert approaches," *Data Analytics for Internet of Things Infrastructure*, pp. 247–264, 2023.

[18] X. Yang, S. Wang, Y. Li, and S. Wang, "Does data sampling improve deep learning-based vulnerability detection? yeas! and nays!" in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2287–2298.

[19] C. Pathania, J. Vijay *et al.*, "The iot solution for reliable transformer condition monitoring and maintenance," in *2023 8th International Conference on Communication and Electronics Systems (ICCES)*. IEEE, 2023, pp. 346–353.

[20] A. Balan, K. Deepa *et al.*, "Detection and analysis of faults in transformer using machine learning," in *2023 International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT)*. IEEE, 2023, pp. 477–482.

[21] X. Nie, N. Li, K. Wang, S. Wang, X. Luo, and H. Wang, "Understanding and tackling label errors in deep learning-based vulnerability detection (experience paper)," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 52–63.